

DOCUMENTO DE DESIGN DE SOFTWARE

FRETE FECHADO ROTAS DE MINAS

Especificação do protótipo digital para implementação assistida pelo Codex

Campo	Definição
Tipo de documento	Software Design Document (SDD)
Versão	1.1 - Protótipo funcional Django + React
Plataforma-alvo	Web/Desktop via navegador, multiplayer local hot-seat
Base de design	GDD Frete Fechado: Rotas de Minas - versão atualizada
Objetivo	Transformar as regras do boardgame em uma implementação digital modular, testável e configurável.

Nota de escopo. Este SDD preserva as regras do GDD como fonte de verdade. Decisões de software que não existem no GDD são explicitamente apresentadas como recomendações de implementação para o protótipo, sem alterar as regras de jogo.

Resumo executivo

O protótipo digital reproduz uma partida local de Frete Fechado para 2 a 4 jogadores. Cada participante administra uma transportadora, aceita uma carga por vez, escolhe rotas entre cinco polos mineiros e gerencia dinheiro, combustível, risco e tempo ao longo de 15 rodadas. O sistema deve ser determinístico quando uma semente aleatória for informada, permitindo testes de balanceamento e reprodução de bugs.

Decisão	Especificação do protótipo
Motor/stack	Frontend React + TypeScript + Vite; backend Django + Django REST Framework; PostgreSQL para persistência. Sem game engine.
Arquitetura	Backend autoritativo com camada de domínio/serviços; API REST; React orientado a estado com Zustand; tabuleiro em SVG/DOM; FSM de turno no backend; dados de balanceamento em JSON.
Persistência	GameSession persistida no backend via Django ORM. PostgreSQL recomendado; SQLite pode ser usado apenas no desenvolvimento local inicial.
Multiplayer	MVP local hot-seat no mesmo navegador. A separação cliente/servidor prepara evolução futura para multiplayer online com Django Channels + Redis.
Conteúdo	Mapa, contratos, eventos, combustível, postos, pneus, serviço local, guincho, 15 rodadas e cálculo de vitória.
Qualidade	pytest/pytest-django para domínio e API; Vitest/React Testing Library para frontend; contratos de API e validação de dados configuráveis.

1. Visão Geral da Arquitetura do Protótipo

1.1 Escopo e objetivo principal

O objetivo é construir um protótipo digital jogável que permita validar o loop principal do boardgame, testar balanceamento e servir como base para evolução futura. O foco é fidelidade às regras, clareza visual e facilidade de alteração de parâmetros, não produção comercial, multiplayer online ou efeitos audiovisuais avançados.

- Partida para 2, 3 ou 4 jogadores locais, alternando turnos no mesmo dispositivo.
- Mapa com cinco polos: Grande BH, Triângulo Mineiro, Sul de Minas, Zona da Mata e Norte de Minas.
- Rotas com quantidade de casas, tipo de estrada, dado, casas de evento, pontos de consumo e postos.

- Contratos públicos por origem, uma carga ativa por jogador e pagamento automático ao chegar ao destino.
- Economia com saldo, serviço local, pneu reserva, vale-combustível, abastecimento e pane seca.
- Encerramento obrigatório após a rodada 15 e ranking por saldo; desempate por entregas.

1.2 Fora do escopo do MVP

- Multiplayer online, login, matchmaking e servidor autoritativo.
- IA adversária, campanha, progressão persistente ou metajogo.
- Editor visual de mapa, monetização, telemetria remota e achievements.
- Simulação geográfica real de Minas Gerais; as rotas são esquemáticas.

1.3 Stack web recomendada (sem game engine)

Camada	Tecnologia	Justificativa
Backend	Python + Django	Centraliza regras, estado da partida, persistência e validação das ações.
API	Django REST Framework	Expõe casos de uso do jogo por endpoints REST e mantém o servidor como fonte de verdade.
Frontend	React + TypeScript	Interface reativa, componentes reutilizáveis e tipagem forte para o estado recebido da API.
Build frontend	Vite	Ambiente de desenvolvimento rápido e bundle web simples.
Estado frontend	Zustand	Store leve para snapshot da partida, estados de UI e ações assíncronas da API.
Persistência	PostgreSQL + Django ORM	Persistência robusta de partidas e histórico; SQLite é aceitável apenas para desenvolvimento inicial.
Renderização do mapa	SVG + React/CSS	Adequado a um boardgame baseado em nós, rotas e casas; permite animações sem engine.
Testes	pytest + pytest-django; Vitest + RTL	Cobertura separada para regras/API e componentes/estado do frontend.

Recomendação arquitetural: o Django deve ser a fonte de verdade da partida. Toda ação do jogador é enviada pelo React como uma intenção para a API; o backend valida a fase do turno, executa a regra e retorna um novo snapshot. O React não implementa regras de negócio: ele apresenta o tabuleiro em SVG/DOM, mantém o snapshot em Zustand e anima as mudanças recebidas do servidor. Essa separação evita divergência de estado e permite evoluir o mesmo backend para multiplayer online no futuro.

1.4 Padrão de arquitetura

```

frete-fechado/
  backend/
    config/          # settings/urls do Django
    game/
      domain/        # entidades, value objects e regras puras
      services/       # movimento, combustível, contratos, eventos, turnos
      api/            # views, serializers, urls e DTOs REST
      repositories/   # acesso ao estado persistido
      data/           # JSON de regras, mapa, eventos e contratos
      models.py       # modelos de persistência Django
      tests/          # pytest/pytest-django
  frontend/
    src/
      components/     # board, cards, HUD, modais
      pages/          # SetupPage, GamePage, GameOverPage
      store/           # Zustand
      services/        # gameApi / cliente HTTP
      types/           # DTOs TypeScript
      hooks/
      public/assets/   # imagens e áudio

```

- State Pattern para `GameState` global: Setup, Playing, GameOver.
- FSM de turno para impedir ações fora de ordem: StartTurn, CityDecision, RouteSelection, Rolling, Moving, ResolveLanding, EndTurn, SkippedTurn.
- A API REST é a fronteira entre regras e apresentação. O backend pode retornar uma lista de `domainEvents` junto ao snapshot para o React executar animações e feedbacks, sem colocar regras em callbacks visuais.
- Cada endpoint deve mapear para um caso de uso/command explícito no backend, facilitando validação, testes, auditoria de jogadas e futuro replay.

1.5 Core Loop traduzido em lógica de programação

1. Iniciar turno do jogador atual e verificar `skipTurnsRemaining`.
2. Se estiver em cidade: permitir aceitar contrato (se vazio), comprar pneu, abastecer em posto urbano e escolher entre Viajar ou Serviço Local.
3. Se escolher Serviço Local: adicionar 2 moedas e encerrar turno.
4. Se escolher Viajar: selecionar uma rota válida a partir da posição atual e manter o sentido até a próxima cidade.
5. Determinar o dado pela rota: D8 para Rodovia; D4 para Secundária/Atalho.
6. Aplicar modificadores pendentes, como Desvio Obrigatório (-2, mínimo 1).
7. Mover casa a casa. Para cada ponto de consumo atravessado, tentar consumir 1 combustível.
8. Se combustível for insuficiente: executar Pane Seca, aplicar guincho (-3 moedas), retornar ao último posto, marcar perda do próximo turno e encerrar o movimento.
9. Se atravessar/alcançar posto rodoviário: oferecer parada voluntária. Se aceita, terminar o movimento no posto e abrir abastecimento.
10. Se terminar numa casa de evento, comprar e resolver exatamente um evento.
11. Se alcançar uma cidade: descartar excesso do dado; se for destino da carga, concluir entrega e receber pagamento automaticamente.

12. Encerrar turno, passar prioridade ao próximo jogador; após todos jogarem, avançar rodada. Ao fim da rodada 15, calcular resultado final.

2. Casos de Uso (Use Cases) e Lógica do Sistema

2.1 Atores

Ator	Responsabilidade
Jogador	Seleciona ações, contratos, rotas, abastecimento e itens pela interface React.
React UI	Renderiza tabuleiro/HUD, coleta input e envia intenções; nunca altera regras ou estado persistente diretamente.
Zustand Store	Mantém o snapshot atual e estados efêmeros de UI; é atualizado apenas pelas respostas da API.
GameApplicationService	Orquestra casos de uso, transações e persistência da GameSession no Django.
TurnController	Valida e executa a FSM do turno.
Rule/Domain Services	Aplicam movimento, combustível, eventos, contratos e vitória.
RNGService	Realiza lançamentos e embaralhamentos reproduzíveis por seed.
GameRepository	Carrega e persiste a sessão via Django ORM.

Caso de uso	Fluxo principal	Pós-condição	Falhas/Exceções
UC-01 - Inicializar partida	Selecionar 2-4 jogadores, criar estado inicial, embaralhar decks, revelar um contrato por polo, posicionar todos em BH, dar 10 moedas, 5 combustíveis e iniciar rodada 1.	`GameSession` válido no estado `Playing`.	Número de jogadores inválido; dados JSON inválidos.
UC-02 - Aceitar contrato	Em uma cidade, se o jogador estiver sem carga, selecionar o contrato público daquela origem; mover a carta para o jogador e repor imediatamente a oferta.	Jogador passa a ter `activeContractId`.	Jogador já possui contrato; não está na origem; pilha e descarte vazios.
UC-03 - Selecionar rota	Listar arestas conectadas à posição atual/cidade seguinte e registrar a rota e direção escolhidas.	`activeRouteTraversal` criado.	Rota não conecta a posição atual ou seleção fora da fase permitida.
UC-04 - Rolar e mover	Escolher D8 ou D4	Posição atualizada;	Pane seca pode

	conforme a rota, aplicar modificadores e mover passo a passo.	combustível/eventos/postos processados.	interromper o movimento.
UC-05 - Abastecer	Em posto urbano, antes da ação principal; ou em posto rodoviário após parada voluntária. Comprar 1..N unidades até o máximo 5. Vale-combustível pode fornecer até 2 unidades sem custo.	Tanque e saldo atualizados.	Quantidade inválida; sem posto; tanque cheio; saldo pode ficar negativo somente por guincho, não por compra normal.
UC-06 - Resolver evento	Ao terminar movimento em casa de evento, comprar a carta superior e aplicar efeito.	Estado alterado e carta descartada.	Nunca comprar evento apenas por atravessar a casa.
UC-07 - Serviço local	Em cidade, escolher não viajar e receber +2 moedas.	Saldo +2 e turno encerrado.	Indisponível em estrada/posto intermediário.
UC-08 - Pane seca	Ao cruzar ponto de consumo com combustível 0, cobrar 3 moedas, mover ao último posto, `skipTurnsRemaining=1`.	Movimento interrompido e penalidade registrada.	Nenhuma; saldo negativo é permitido para o guincho.
UC-09 - Concluir entrega	Ao chegar à cidade destino do contrato ativo, adicionar pagamento, incrementar entregas e remover contrato ativo.	Jogador fica sem carga e pode aceitar outra carga local; movimentação somente no próximo turno.	Não se aplica se cidade não for o destino.
UC-10 - Encerrar partida	Após o último turno da rodada 15, aplicar -3 por contrato pendente, ordenar por saldo e desempatar por entregas.	Estado `GameOver` com ranking final.	Empate total resulta em vitória compartilhada.
UC-11 - Salvar/retomar	Persistir `GameSession`, seed, estado do RNG e versão no backend. Ao abrir uma partida existente, carregar pelo ID e retornar um snapshot validado pela API.	Partida retomada exatamente do mesmo estado persistido.	Registro inexistente, versão incompatível ou sessão corrompida -> responder erro estruturado e não iniciar estado parcial.

2.2 Fluxo de dados entre componentes

Input do jogador

- > Componente React
- > action da Zustand Store
- > gameApi.ts (HTTP/JSON)
- > DRF View + Serializer
- > GameApplicationService / UseCase

- > Domain Services + TurnController
- > GameRepository (Django ORM)
- > GameSession persistida
- > Response { snapshot, domainEvents, legalActions }
- > Zustand Store
- > Board SVG / HUD / Modal

JSON de configuração no backend

- > ConfigLoader/Validator
- > Catalogs de rotas/eventos/contratos
- > Domain Services

Endpoints mínimos:

- POST /api/games/
- GET /api/games/{gameId}/
- POST /api/games/{gameId}/actions/accept-contract/
- POST /api/games/{gameId}/actions/select-route/
- POST /api/games/{gameId}/actions/roll-move/
- POST /api/games/{gameId}/actions/refuel/
- POST /api/games/{gameId}/actions/local-service/
- POST /api/games/{gameId}/actions/end-turn/
- POST /api/games/{gameId}/reset/

A UI envia intenções, nunca alterações diretas. Cada endpoint retorna um DTO estruturado com `ok`, `snapshot`, `legalActions`, `domainEvents` e, quando aplicável, `errors`. O frontend substitui/normaliza seu snapshot a partir da resposta do servidor. Para ações mutáveis, o backend deve usar transação e bloquear/validar a versão da sessão para evitar dupla execução acidental.

2.3 Regras de precedência durante movimento

- Avançar para a próxima casa.
- Registrar passagem por posto; se for um posto, atualizar `lastVisitedStationId`.
- Se a casa/ponto possui consumo, tentar consumir combustível. Se falhar, pane seca ocorre imediatamente e os passos restantes são descartados.
- Se a casa é posto rodoviário e ainda existem passos, a UI pode oferecer parada voluntária. Se aceitar, movimento encerra ali.
- Ao alcançar uma cidade, movimento termina imediatamente; passos restantes são descartados.
- Somente depois de determinar a posição final, se ela for uma casa de evento, resolver o evento.

Decisão de implementação. A ordem acima elimina ambiguidades de software sem mudar a intenção do GDD: consumo acontece ao atravessar o ponto; eventos só ao parar; cidade encerra o movimento; posto pode encerrar o movimento voluntariamente.

3. Especificação dos Sistemas de Software

3.1 Modelo de domínio e classes principais

Classe/Serviço	Responsabilidade	Invariantes / notas
GameSession	Estado completo da partida: round,	Entidade/aggregate de domínio

	currentPlayerIndex, players, board, decks, publicContracts, phase, winnerResult.	serializável; persistida pelo repositório e nunca mutada diretamente pela UI.
PlayerState	id, displayName, money, fuel, location, activeContractId, spareTire, fuelVoucher, deliveries, pendingEffects, skipTurnsRemaining.	Clamp de combustível em 0..5; dinheiro pode ser negativo apenas quando regras permitirem.
BoardGraph	cities, routes, spaces, stations, adjacency.	Consulta rotas conectadas e resolve posição/direção.
RouteDefinition	id, fromCityId, toCityId, steps, roadType, dieSides, spaces.	roadType determina D8/D4; espaços carregam tags.
SpaceDefinition	index, tags: EVENT/FUEL_CONSUMPTION/STATION/CITY.	Pode conter mais de uma tag somente se configurado explicitamente.
ContractDefinition	id, originCityId, destinationCityId, reward, cargoTheme.	Uma carta ativa por jogador.
EventCardDefinition	id, type, weight/count, effect payload, text.	Efeito interpretado por EventResolver.
GameApplicationService	Orquestra endpoints/casos de uso, transações, GameSession, TurnController e resposta DTO.	Único ponto de entrada para mutações vindas da API.
TurnController	FSM do turno; expõe ações permitidas por fase.	Impede input ilegal.
MovementService	Calcula dado, modificadores e avanço passo a passo.	Produz `domainEvents` para o frontend animar; não conhece React/SVG.
FuelService	Consumo, compra, vale e pane seca.	Mantém `lastVisitedStationId`.
ContractService	Aceite, reposição, entrega e penalidade final.	Reembaralha descarte quando necessário.
EventResolver	Aplica os seis tipos de evento do GDD.	Nenhum evento compra outro evento.
VictoryService	Aplica penalidades e ranking final.	Desempate por entregas; empate restante é compartilhado.
RNGService	roll(sides), shuffle(list), getState().	Seed injetável para reprodutibilidade.
GameRepository	save/load/delete com Django ORM.	Adapter de persistência; domínio não conhece Django nem banco.

3.2 Estados do jogo (FSM global)

Estado	Entrada	Ações válidas	Saída
BOOT	Frontend carregado e	Carregar assets, verificar	SETUP

	backend acessível	health/config e inicializar store	
SETUP	Dados válidos	Escolher 2-4 jogadores, seed opcional e criar partida via POST /api/games/	PLAYING
PLAYING	Sessão criada	Executar ações via API, atualizar snapshot, abrir ajuda	GAME_OVER após rodada 15
GAME_OVER	Resultado calculado	Ver ranking, nova partida	SETUP
ERROR	Falha de API/config ou estado irrecuperável	Mostrar erro técnico, permitir retry/reload ou nova partida	SETUP/BOOT

3.3 Estados do turno (FSM de turno)

Fase	Descrição	Ações permitidas
START_TURN	Aplica perda de turno e prepara contexto.	Continue ou skip automático.
CITY_DECISION	Jogador em cidade antes da ação principal.	Aceitar contrato, comprar pneu, abastecer se houver posto; escolher Viajar ou Serviço Local.
ROUTE_SELECTION	Escolha da conexão/rota.	Selecionar rota válida; cancelar para CityDecision.
ROLLING	Lançamento lógico e apresentação.	Confirmar/animar rolagem.
MOVING	Movimento casa a casa.	Somente prompts de posto quando aplicável.
REFUELING	Parada em posto.	Comprar combustível/usar vale; finalizar.
RESOLVE_LANDING	Cidade, evento ou posição final.	Resolver evento/entrega; sem novo movimento.
END_TURN	Commit do turno e logs.	Avançar jogador/rodada.
SKIPPED_TURN	Perda de turno por guincho.	Decrementar contador e encerrar.

3.4 Estruturas de dados configuráveis

Todos os valores de balanceamento devem ficar fora do código sempre que possível. Abaixo está o formato recomendado; os nomes podem ser adaptados, mas o significado deve permanecer.

```
// game-rules.json
{
  "maxRounds": 15,
  "startingMoney": 10,
  "startingFuel": 5,
```

```

"fuelCapacity": 5,
"fuelUnitPrice": 1,
"localServiceReward": 2,
"spareTirePrice": 2,
"towCost": 3,
"towSkipTurns": 1,
"pendingContractPenalty": 3,
"fuelVoucherUnits": 2
}

// routes.json (exemplo)
{
  "id": "bh_norte_highway",
  "fromCityId": "bh",
  "toCityId": "norte",
  "steps": 9,
  "roadType": "HIGHWAY",
  "dieSides": 8,
  "eventSpaces": [4],
  "fuelConsumptionSpaces": [4, 8],
  "stationSpaces": [5]
}

```

3.5 Parâmetros iniciais do mapa

Ligação	Passos	Tipo	Dado	Eventos	Consumo
Triângulo - BH	8	Rodovia	D8	2	1
Triângulo - Sul	7	Secundária	D4	2	1
Sul - BH	6	Rodovia	D8	1	1
Sul - Zona da Mata	7	Secundária	D4	2	1
BH - Zona da Mata	5	Rodovia	D8	1	1
BH - Norte / rodovia	9	Rodovia	D8	1	2
BH - Norte / atalho	6	Atalho	D4	3	1

Postos do primeiro protótipo: urbanos em Grande BH, Triângulo Mineiro e Sul de Minas; intermediários na rodovia BH-Norte e no eixo Sul-Zona da Mata. Norte de Minas e Zona da Mata não possuem posto urbano na versão inicial.

3.6 Economia e contratos

Elemento	Valor inicial
Saldo inicial	10 moedas
Tanque inicial/máximo	5 unidades
Combustível	1 moeda/unidade
Serviço local	+2 moedas

Pneu reserva	2 moedas
Guincho	-3 moedas + perde próximo turno
Contrato pendente ao final	-3 moedas

Frete	Pagamento
BH - Zona da Mata	7
BH - Sul	8
Triângulo - Sul	9
Sul - Zona da Mata	9
Triângulo - BH	10
BH - Norte	11

Os pagamentos valem nos dois sentidos, conforme o GDD. Para o protótipo, recomenda-se gerar pelo menos 2 a 4 cartas por par de origem/destino para permitir embaralhamento e variação visual sem alterar o valor econômico.

3.7 Eventos e efeitos

Evento	Qtd.	Implementação
Pneu furado	3	Se `spareTire=true`, consumir o item; caso contrário `money -= 3`.
Reparo mecânico	2	`money -= 2`.
Desvio obrigatório	2	Adicionar `nextMovePenalty = 2`; próximo resultado = $\max(1, \text{roll} - 2)$; consumir efeito.
Fiscalização de rotina	1	Sem efeito mecânico; registrar log/UI.
Assistência da seguradora	2	`money += 1`.
Vale-combustível	2	Se não possui vale, `fuelVoucher=true`; senão descartar carta sem efeito. No próximo posto, até 2 unidades gratuitas.

3.8 Regras e invariantes críticas

- Cada jogador pode possuir no máximo um contrato ativo, um pneu reserva e um vale-combustível.
- Fuel sempre permanece entre 0 e 5.
- Eventos são ativados somente se o movimento terminar na casa de evento.

- Consumo de combustível ocorre ao atravessar o marcador, independentemente de terminar nele.
- Chegar a uma cidade encerra o movimento e descarta passos excedentes.
- Parada voluntária em posto rodoviário encerra o movimento daquele turno.
- Serviço local só pode ser usado em cidade.
- Pane seca encerra imediatamente o movimento e define perda do próximo turno.
- Entrega é automática ao alcançar a cidade destino.
- Após aceitar uma nova carga na chegada, o jogador só pode voltar a se mover no turno seguinte.
- A partida termina exatamente após todos jogarem na rodada 15.

3.9 Tratamento de erros e validações

Categoria	Tratamento
Input ilegal	Backend retorna HTTP 400/409 com `{ ok:false, code, message }`; transação não persiste mudança.
JSON inválido	Falhar na inicialização do backend/config loader, registrar arquivo/campo e impedir criação de partidas até correção.
Deck vazio	Reembaralhar descarte; se ambos vazios, deixar oferta vazia até uma entrega devolver carta.
Sessão incompatível	Validar `schemaVersion`; rejeitar snapshot antigo sem migração automática no MVP.
Estado impossível	Lançar `DomainInvariantError`; fazer rollback da transação, registrar gameId/snapshot e retornar erro 500 controlado.
RNG	Nunca usar RNG não injetável dentro das regras; `RNGService` deve ser seedable e testável no backend.

3.10 Estratégia de testes

- Unitários: movimento, consumo, pane seca, pagamento de contratos, eventos, desempate e reposição de decks.
- Testes de propriedade simples: combustível nunca >5 ou <0 ; jogador nunca possui >1 contrato; rodada nunca >15 em Playing.
- Integração: fluxo completo de um turno de cidade até evento/entrega; guincho + skip; fim da rodada 15.
- Seed fixa para cenários reproduzíveis.
- Teste manual de UX com 2, 3 e 4 jogadores e registro dos indicadores previstos no GDD: duração, saldo, entregas, eventos, serviços locais, rotas, combustível, abastecimentos, paradas e panes.

4. Lista de Assets Recomendados para o Protótipo

A proposta visual do GDD é um tabuleiro claro, com fundo predominantemente branco, contornos nítidos, casas retangulares, faixas de cor e ícones dos produtos. O protótipo deve priorizar legibilidade; os assets abaixo podem ser placeholders.

4.1 Sprites e elementos de tabuleiro

Arquivo	Categoria	Formato/dimensão	Uso
spr_city_bh.png	Cidade/ícone	256x256 PNG transparente	Grande BH / indústria
spr_city_triangulo.png	Cidade/ícone	256x256 PNG transparente	Triângulo / grãos
spr_city_sul.png	Cidade/ícone	256x256 PNG transparente	Sul / café
spr_city_zona_mata.png	Cidade/ícone	256x256 PNG transparente	Zona da Mata / laticínios
spr_city_norte.png	Cidade/ícone	256x256 PNG transparente	Norte / agrícolas/tecidos
spr_truck_p1.png ... spr_truck_p4.png	Meeple/caminhão	128x128 PNG transparente	Uma cor por jogador
spr_space_normal.png	Casa comum	96x96 PNG	Base neutra
spr_space_event.png	Casa de evento	96x96 PNG	Símbolo !
spr_space_station.png	Posto	96x96 PNG	Bomba de combustível
spr_space_fuel_tick.png	Ponto de consumo	64x64 PNG	Marcador simples de consumo
spr_route_highway.png	Trecho de rodovia	256x64 PNG/tile	Identidade de rodovia
spr_route_secondary.png	Trecho secundário	256x64 PNG/tile	Identidade de rota lenta/atalho

4.2 UI e HUD

Arquivo	Elemento	Formato/dimensão
ui_panel_player.png	Painel de jogador	9-slice PNG, 640x240
ui_coin.png	Moeda	64x64 PNG
ui_fuel_full.png	Combustível cheio	64x64 PNG
ui_fuel_empty.png	Combustível vazio	64x64 PNG
ui_tire.png	Pneu reserva	64x64 PNG
ui_voucher.png	Vale combustível	64x64 PNG
ui_contract_card.png	Moldura contrato	512x768 PNG
ui_event_card.png	Moldura evento	512x768 PNG
ui_die_d4.png / ui_die_d8.png	Ícone dado	128x128 PNG
ui_round_marker.png	Rodada	64x64 PNG
ui_btn_primary.png	Botão	9-slice PNG, 320x96

ui_modal.png	Modal	9-slice PNG, 800x520
--------------	-------	----------------------

4.3 Áudio

Arquivo	Formato	Uso
sfx_dice_roll.wav	WAV 44.1kHz	Rolagem de D4/D8
sfx_move_step.wav	WAV 44.1kHz	Movimento de casa
sfx_coin_gain.wav	WAV 44.1kHz	Receita/serviço/entrega
sfx_coin_loss.wav	WAV 44.1kHz	Despesa
sfx_refuel.wav	WAV 44.1kHz	Abastecimento
sfx_event.wav	WAV 44.1kHz	Carta de evento
sfx_tow.wav	WAV 44.1kHz	Pane seca/guincho
sfx_delivery.wav	WAV 44.1kHz	Entrega concluída
bgm_main_loop.ogg	OGG estéreo	Loop leve opcional; volume baixo

4.4 Animações e VFX mínimos

- Animação do caminhão entre casas: 120-180 ms por casa usando CSS transforms/Web Animations API ou Framer Motion opcional, com opção de acelerar/desativar.
- Animação simples do dado no React: 500-800 ms antes de revelar o valor lógico já determinado pelo backend/RNGService.
- Feedback de combustível: marcador pisca e desaparece ao consumir; preenchimento ao abastecer.
- Toast de ganho/perda de moedas.
- Modal de evento e modal de pane seca.
- Sem partículas obrigatórias. Confete discreto na tela final é opcional.

4.5 Organização de assets

```
public/assets/
  board/
  cities/
  trucks/
  ui/
  cards/
  audio/sfx/
  audio/bgm/
```

5. Diretrizes e Prompt de Contexto para o Codex

5.1 Ordem recomendada de implementação

19. Criar monorepo simples com `backend/` Django + DRF e `frontend/` React + TypeScript + Vite. Configurar pytest/pytest-django no backend e Vitest/React Testing Library no frontend.
20. Criar entidades/value objects Python do domínio e DTOs/serializers da API; espelhar os DTOs públicos em tipos TypeScript no frontend.
21. Criar JSONs de regras, mapa, eventos e contratos no backend; validar no startup/testes antes de aceitar novas partidas.
22. Implementar `SeededRNGService` no backend e testes determinísticos com pytest.
23. Implementar `BoardGraph` e navegação bidirecional das rotas.
24. Implementar `FuelService` e testes de consumo, abastecimento, vale e pane seca.
25. Implementar `MovementService` com movimento casa a casa, cidade, posto e evento.
26. Implementar `ContractService` com oferta, reposição e entrega.
27. Implementar `EventResolver` para os seis eventos.
28. Implementar `TurnController` e FSM no backend, sem dependência de HTTP/React; validar por testes.
29. Implementar `GameApplicationService` + `GameRepository` para rodadas, 2-4 jogadores, persistência e fim de partida.
30. Criar páginas React: SetupPage, GamePage e GameOverPage; configurar Zustand e cliente HTTP (gameApi.ts).
31. Criar tabuleiro em SVG + componentes React para cidades, rotas, casas e caminhões; usar formas/ícones placeholders antes dos assets finais.
32. Conectar inputs React aos endpoints REST; nenhum componente ou store deve aplicar regras localmente ou mutar o snapshot como fonte de verdade.
33. Persistir partidas com Django ORM e adicionar logs de ações/partida para reprodução de bugs e balanceamento.
34. Executar testes backend/frontend e uma partida manual completa via navegador antes de adicionar polish audiovisual.

5.2 Convenções de código

- Frontend TypeScript em `strict: true`, evitando `any`; backend Python com type hints nas camadas de domínio e serviço.
- Python: classes em PascalCase e funções/variáveis em snake_case. TypeScript/React: componentes/classes em PascalCase e funções/variáveis em camelCase. IDs de domínio em snake_case legível.
- Um arquivo por classe/serviço relevante; evitar arquivos acima de ~300 linhas quando possível.
- Sem números mágicos: valores de balanceamento devem vir de config JSON ou `GameRules`.
- Funções de regra devem ser puras ou ter efeitos controlados e testáveis.
- No backend, representar ações inválidas esperadas com erros de domínio mapeados para respostas DRF 400/409; exceções apenas para invariantes quebradas. No frontend, tratar respostas de erro sem alterar o snapshot válido.

- Não acoplar regras ao Django REST nem ao React: `game/domain` e serviços de regra não importam componentes web; views/serializers apenas adaptam entrada e saída.
- RNG e relógio devem ser dependências injetáveis no backend quando afetarem testes; chamadas HTTP devem ser isoladas em `frontend/src/services`.
- Toda alteração de regra deve vir acompanhada de teste.

5.3 Pontos de extensão futura

Extensão	Preparação no MVP
IA de jogadores	Expor `getLegalActions()` e snapshots imutáveis do estado.
Multiplayer online	Backend já autoritativo; adicionar Django Channels + Redis, autenticação/salas e broadcast de snapshots/eventos sem reescrever regras.
Editor de mapa	Rotas/casas já são data-driven em JSON.
Novas cartas/eventos	Efeitos identificados por `effectType` e payload; resolver por registry.
Novos tipos de veículo	Criar `VehicleDefinition` depois do MVP; não codificar velocidade no PlayerState.
Simulador de balanceamento	Domínio Python independente de Django/React permite executar milhares de partidas headless em scripts/testes.
Replays	Registrar commands/actions + seed no backend; reexecutar sobre estado inicial.

5.4 Prompt de contexto mestre para o Codex

O texto abaixo pode ser fornecido ao Codex no início do projeto. Ele deve ser usado junto com este SDD e os JSONs do protótipo.

Você é o engenheiro responsável por implementar o protótipo digital de "Frete Fechado: Rotas de Minas".

OBJETIVO

Crie um boardgame digital local hot-seat para 2 a 4 jogadores, executado no navegador. Preserve rigorosamente as regras deste SDD. Priorize regras corretas, código modular, testes automatizados e dados de balanceamento configuráveis. NÃO use game engine.

STACK OBRIGATÓRIA

BACKEND

- Python 3.x
- Django
- Django REST Framework
- PostgreSQL (SQLite permitido somente no desenvolvimento inicial)
- pytest + pytest-django

FRONTEND

- React
- TypeScript strict
- Vite
- Zustand

- SVG/DOM/CSS para renderizar e animar o tabuleiro
- Vitest + React Testing Library

ARQUITETURA

- O backend Django é a fonte de verdade da partida.
- `backend/game/domain`: entidades, value objects e regras puras; não depende de DRF.
- `backend/game/services`: casos de uso e serviços de movimento, combustível, contratos, eventos, turno e vitória.
- `backend/game/api`: views, serializers, URLs e DTOs REST.
- `backend/game/repositories`: persistência da GameSession via Django ORM.
- `backend/game/data`: JSON de regras/mapa/eventos/contratos.
- `frontend/src/components`: tabuleiro SVG, HUD, cartas e modais.
- `frontend/src/store`: Zustand com o snapshot retornado pela API e estado efêmero de UI.
- `frontend/src/services`: único local para chamadas HTTP.
- O frontend NUNCA aplica regras de jogo como fonte de verdade.

FLUXO OBRIGATÓRIO

Input React -> Zustand action -> gameApi -> DRF endpoint -> ApplicationService -> Domain Services/FSM -> Repository -> resposta `{ snapshot, legalActions, domainEvents }` -> Zustand -> UI.

REGRAS PRINCIPAIS

- 2 a 4 jogadores, 15 rodadas, todos começam em Grande BH.
- Saldo inicial 10; combustível 5/5; sem carga, pneu ou vale.
- Um contrato ativo por jogador.
- Rodovia usa D8; secundária/atalho usa D4.
- Chegar à cidade encerra movimento e descarta excesso do dado.
- Evento somente ao PARAR numa casa de evento.
- Consumo acontece ao ATRAVESSAR cada ponto de consumo.
- Posto rodoviário permite parada voluntária; posto urbano permite abastecer antes da ação principal.
- Combustível custa 1 moeda/unidade, máximo 5.
- Pane seca: -3 moedas, retorna ao último posto, perde próximo turno; saldo pode ficar negativo por esse custo.
- Serviço local em cidade: +2 moedas e encerra turno.
- Entrega é automática ao chegar ao destino e paga o valor do contrato.
- Contrato pendente no fim: -3 moedas.
- Vitória: maior saldo; desempate por entregas; persistindo empate, vitória compartilhada.

IMPLEMENTAÇÃO

1. Implemente primeiro domínio e serviços Python, sem HTTP e sem React.
2. Nunca use aleatoriedade diretamente nas regras; use `RNGService` seedable.
3. Mantenha parâmetros de balanceamento em JSON no backend; não use números mágicos.
4. Adicione testes pytest para cada regra antes de criar endpoints.
5. Crie endpoints DRF finos: validar input, chamar um caso de uso e serializar resposta.
6. Toda mutação de partida deve ocorrer em transação e retornar o snapshot atualizado.
7. O React deve apenas enviar intenções e renderizar respostas; não duplique regras no frontend.
8. Renderize o mapa com SVG. Cidades, casas, rotas e caminhos devem ser componentes React posicionados por coordenadas/configuração.
9. Use CSS/Web Animations API ou Framer Motion opcional para animações; nenhuma engine é permitida.
10. Use placeholders geométricos se um asset não existir; arte nunca deve bloquear o protótipo.
11. Trate erros de API de forma estruturada e preserve o último snapshot válido.
12. Se o SDD não definir uma regra, não invente silenciosamente: crie TODO explícita e documente a decisão temporária.

CRITÉRIO DE PRONTO

É possível criar uma partida por API, abrir a GamePage, completar 15 rodadas com 2-4 jogadores, aceitar e entregar contratos, escolher rotas, usar D8/D4, consumir/abastecer combustível, sofrer pane seca, resolver eventos, usar serviço local, persistir/retomar a partida, calcular a classificação final e reiniciar, sem inconsistência entre frontend e backend.

5.5 Prompts modulares recomendados

Etapa	Prompt curto para Codex
Módulo 1 - Domínio	Implemente entidades/value objects Python e factories do domínio, sem Django REST e sem React. Inclua pytest para invariantes e configuração.
Módulo 2 - Mapa e movimento	Implemente BoardGraph e MovementService em Python para rotas bidirecionais, D8/D4 e movimento passo a passo. Inclua pytest para chegada com excesso de dado.
Módulo 3 - Combustível	Implemente FuelService, postos, vale, consumo por marcador, lastVisitedStationId e pane seca. Cubra todos os casos com pytest.
Módulo 4 - Cartas	Implemente ContractService e EventResolver; decks devem usar RNGService e reembaralhar descarte quando necessário.
Módulo 5 - Turnos	Implemente TurnController como FSM no backend. Exponha <code>get_legal_actions()</code> para a camada de aplicação/API.
Módulo 6 - React/SVG	Crie SetupPage, GamePage, GameOverPage, Zustand e tabuleiro SVG consumindo somente a API REST. Use shapes/placeholders antes de assets finais.
Módulo 7 - Integração	Crie endpoints DRF e conecte o React. Faça testes de API e uma partida determinística/reduzida; depois execute manualmente 15 rodadas.

Apêndice A - Critérios de aceitação do protótipo

- Usuário consegue configurar 2-4 jogadores e iniciar a partida.
- O jogador atual e a rodada ficam sempre visíveis.
- A UI só habilita ações válidas para o estado atual.
- D8 é usado em rodovias; D4 em secundárias/atalhos.
- Combustível é consumido exatamente nos marcadores cruzados.
- Abastecimento respeita posto, custo 1 e capacidade 5.
- Pane seca aplica custo 3, retorno ao último posto e perda de 1 turno.
- Eventos só disparam ao parar e aplicam os efeitos descritos.
- Contratos podem ser aceitos apenas na origem e com caminhão vazio.
- Entrega automática paga corretamente e incrementa o contador.
- Serviço local adiciona 2 moedas e encerra o turno.
- Rodada 15 encerra sem turnos extras; contrato pendente recebe -3.
- Ranking final aplica desempate por entregas e admite empate compartilhado.
- Nova partida limpa completamente o estado anterior.
- Testes automatizados do domínio passam no CI/local.

- O backend é a única fonte de verdade: atualizar ou recarregar a página não altera regras nem duplica ações já persistidas.
- O tabuleiro funciona em React/SVG sem game engine, Unity, Godot ou outra game engine.
- Endpoints inválidos ou repetidos não deixam a GameSession em estado parcial; testes de API cobrem os principais casos de uso.

Apêndice B - Premissas e pontos que devem permanecer configuráveis

O GDD declara os valores atuais como parâmetros iniciais sujeitos a testes. Portanto, o protótipo deve tratar como dados configuráveis: número de rodadas, dinheiro inicial, capacidade/consumo de combustível, preços, valores de eventos, número e posição de casas, dados por tipo de estrada, localização dos postos e pagamentos dos fretes. Nenhum desses itens deve exigir alteração estrutural no código para ser rebalanceado.